

Session 1 : Examen de Fin de Semestre JavaScript (M. Gueye)

Exercice 1 : Manipulation du DOM et Requêtes AJAX avec jQuery

(8 points)

On considère une page HTML intégrant un tableau d'identifiant unique et de classe associés : `<table id="tabid1" class="tab1">`. Son bloc `<tbody>` liste de manière dynamique un ensemble d'ouvrages littéraires. Chaque ligne de données représente un livre modélisé par les attributs : **id**, **titre**, **auteur**, **prix**, **année**, **thème**. Une liste vide d'identifiant `<ul id="listeLivres"></ul>` est disponible.

1. Écrire le script jQuery permettant d'extraire l'intégralité du contenu textuel et des attributs des éléments du `tbody` (**Contrainte stricte** : Interdiction formelle de faire usage des méthodes `forEach`, `push` ou `find`). (1.5 pts)
2. Sauvegarder de manière persistante cette collection de livres au sein de l'espace de stockage de l'utilisateur via l'objet `localStorage`. (1.5 pts)
3. Développer l'appel asynchrone jQuery `$.ajax()` permettant de transmettre en méthode `POST` les données d'inventaire vers le point d'accès API local distant : `https://localhost:3000/livres`. (1.5 pts)
4. Injecter et afficher proprement chaque élément du jeu de données au sein de la liste HTML `<ul>` en faisant figurer uniquement le titre, l'auteur et l'année de parution. (1.5 pts)
5. Associer dynamiquement à chaque ligne générée deux éléments d'action de type bouton intitulés respectivement **Supprimer** et **Modifier**, et programmer la gestion d'événements jQuery correspondante. (2 pts)

Exercice 2 : Composants Applicatifs et Cycle de vie sous React

(6 points)

Concevoir un composant fonctionnel React autonome nommé `EtudiantChecker` qui réalise pas-à-pas les traitements de données d'API suivants :

1. Interroger et récupérer la liste complète des utilisateurs via l'API publique de test : `https://jsonplaceholder.typicode.com/users`
2. Stocker immédiatement l'intégralité du tableau de données JSON récupéré au sein du `localStorage` du navigateur.
3. Parcourir la structure de données locale à la recherche spécifique de l'utilisateur dont la propriété `name` est égale à "Leanne Graham" et le champ `email` correspond à "Sincere@april.biz".
4. Si l'utilisateur recherché existe, charger ses attributs au sein d'un formulaire éditable présentant deux champs d'entrée contrôlés : Nom (`name`) et Adresse Email (`email`).
5. Assurer la mise à jour réactive des états du composant lors de la saisie de l'utilisateur au clavier via le gestionnaire `onChange`.
6. Ajouter un bouton de validation de formulaire qui exécute l'envoi des données modifiées à l'API via la méthode globale jQuery `$.post`.

Contraintes de développement :

Pour le traitement et le parcours de vos structures de données, l'utilisation des méthodes modernes de tableau de type `map()`, `forEach()`, `filter()`, `find()` ou `$.each()` est interdite. Vous devez obligatoirement implémenter des structures de boucles algorithmiques classiques (`for` / `while`).

Exercice 3 : Traitement Algorithmique et Filtrage de Flux Dynamiques

(6 points)

Soit une interface web dotée d'une table HTML de structure `<table id="tabLivres">` présentant les champs ID, Titre, Auteur et Prix. L'interface comprend également un champ de saisie numérique `<input id="prixMin">`, un bouton d'action `<button id="filtrer">Filtrer</button>` ainsi qu'un conteneur de liste réceptacle `<ul id="livresFiltres"></ul>`.

Développer au choix en JavaScript natif (VanillaJS) ou en jQuery les scripts permettant de :

1. Capturer les lignes du tableau HTML et instancier une structure de données de type tableau d'objets (chaque objet représentant un livre avec ses propriétés d'ID, Titre, Auteur et Prix) en utilisant une structure de boucle classique.
2. Écouter le clic sur le bouton de filtrage pour analyser le tableau d'objets et insérer dans la liste `<ul>` les livres (2 pts)
dont le prix est supérieur ou égal au montant indiqué dans le champ de saisie. (2 pts)
3. Adjoindre à chaque élément inséré un bouton fonctionnel de suppression permettant de retirer instantanément l'élément parent de l'arborescence HTML de l'interface graphique lors du clic. (2 pts)

Session 2 : Épreuve de Rattrapage JavaScript & Frameworks

Exercice 1 : Intégration et Syntaxe JSX sous React

(6 points)

- Expliquer la méthodologie technique standard permettant d'embarquer et de faire interpréter du code de syntaxe JSX au sein d'une page web HTML5 classique.
- Développer une fonction JavaScript minimale renvoyant un élément JSX structuré affichant le message textuel "Hello".
- Au niveau de l'appel de rendu de l'application via la méthode globale `ReactDOM.render(...)`, ajouter et exploiter une propriété personnalisée (prop) sur cette fonction.

Exercice 2 : Algorithme de Validation de Formulaire

(6 points)

Rédiger le script de programmation JavaScript complet permettant d'assurer le contrôle strict de sécurité d'un formulaire d'envoi avant sa soumission. Le traitement doit impérativement remplir les exigences fonctionnelles suivantes :

- Bloquer instantanément le déclenchement de l'envoi si le champ d'adresse de messagerie électronique (email) est laissé vide.
- Vérifier la conformité syntaxique de l'adresse email via une expression régulière (Regex) adaptée.
- Injecter dynamiquement un message visuel de confirmation de réussite au sein de l'interface après validation complète des critères de saisie.

Exercice 3 : Événements Multiples et Réponses d'Interface avec jQuery

(4 points)

Écrire le bloc de code HTML5 d'insertion d'une image dotée d'un attribut d'identifiant unique. Développer ensuite le script d'écoute jQuery permettant de lier et de gérer de manière distincte les trois événements système suivants appliqués à cette image :

- `click` : Dès le clic simple de l'utilisateur sur l'image, appliquer instantanément des styles de bordures spécifiques à l'élément.
- `dblclick` : Lors d'un double-clic sur l'élément, modifier dynamiquement la couleur d'arrière-plan (background-color) de l'élément.
- `load` : Programmer le script pour qu'au chargement complet initial du composant dans la page, la couleur de fond par défaut soit initialisée en jaune.

Exercice 4 : Persistance Locale et Tableaux HTML Dynamiques

(4 points)

L'interface de l'API web `localStorage` permet un stockage persistant de données sous forme de paires clé/valeur au format chaîne de caractères. On rappelle les syntaxes d'accès fondamentales :

- Lecture et décodage de données : `let items = JSON.parse(localStorage.getItem("key"));`
- Codage et sérialisation des données : `localStorage.setItem("key", JSON.stringify(items));`

Concevoir un formulaire d'inscription complet. Développer le script JavaScript associé permettant d'enregistrer chaque nouveau profil utilisateur au sein du stockage local, puis de rafraîchir dynamiquement un tableau HTML récapitulatif nommé "Liste des visiteurs".